

CAPTCHA Recognition System

Tu Jia En
Guo Yuhao
Ong Sheng Jin Owen

Ho Jia Cheng
Yek Khoon Sun

Contributions:
Tu Jia En: Data Cleaning, CNN, OCR-style with BiLSTM CTC
Guo Yuhao: Data Cleaning, Preprocessing, Segmentation (DBSCAN), CNN
Ong Sheng Jin Owen: Data Cleaning, GAN
Ho Jia Cheng: Data Cleaning, Segmentation, FasterRCNN
Yek Khoon Sun: Data Cleaning, Segmentation (K-clustering)

Abstract

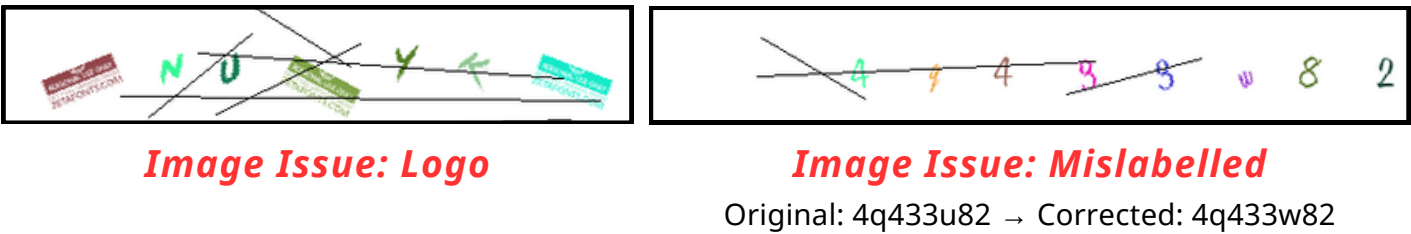
In this project, we design and implement a CAPTCHA recognition system using a convolutional neural network (CNN). The system is built on three main pillars:

- 1.Data Cleaning and Preprocessing:** We curate and preprocess real CAPTCHA data by removing noise, correcting labelling errors, and segmenting the images into individual characters.
- 2.CNN Model:** We train a convolutional neural network using regularisation and data-augmentation techniques to accurately classify each CAPTCHA character.
- 3.Data generation with GAN:** We expand and balance the dataset using a Generative Adversarial Network (GAN) trained to produce realistic CAPTCHA images, providing additional synthetic samples that capture the distortions, overlaps, and noise patterns seen in real CAPTCHAs.

Image Preprocessing Pipeline

Data Cleaning

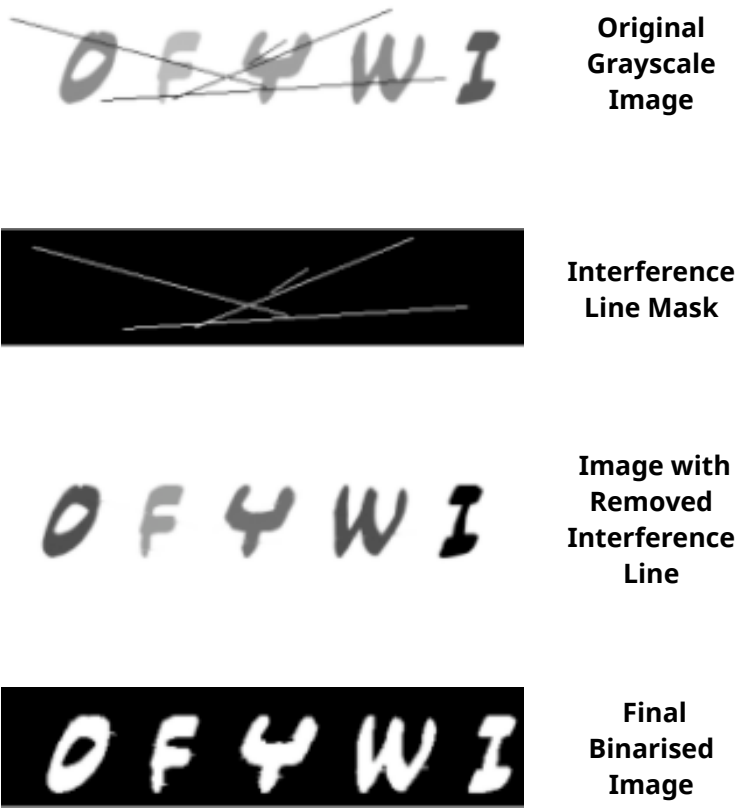
We manually curated the training dataset by removing low-quality images and correcting mislabeled samples to ensure high-quality training data.



Preprocessing

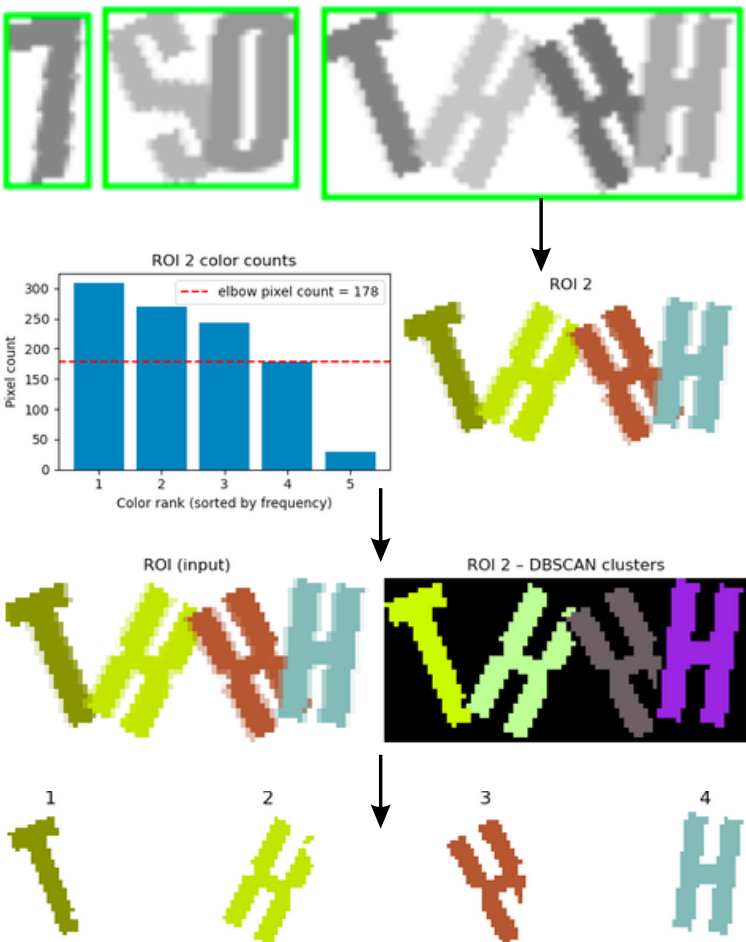
Our preprocessing pipeline focuses on removing interference lines and preparing a clean binary captcha image:

1. Read image as **grayscale**.
2. Build a **dark-pixel mask** where pixel < 5 (these are the interference lines).
3. **Inpaint using that mask** to remove the lines while keeping background/characters.
4. **Binarise with cv2.THRESH_BINARY_INV** to get a clean character mask (white chars on black).



Segmentation (DBSCAN)

1. Identify contours in the image, create bounding boxes.
2. For each bounding box, get the color counts for each unique color.
3. Sort the color counts and find the *elbow*. Where the sharpest dropoff is seen.
4. Use DBSCAN with eps=1 and min_samples = *elbow* (pixel counts) on a color & position weighted vector. Color is significantly more important than position.
5. Apply the non-noise masks and generate segments. Keep matching length only.



Recognition Approaches

Training pipeline

1. Generate a corpus of labelled characters to images. Do a simple 8:2 train-test split.
2. For training, Use Transform to reshape into 32x32 and apply random rotation +/- 30 degrees. Increase model robustness.
3. Ingest individual chars as greyscale into model. Try various models and evaluate with Precision, Recall, Accuracy and F1 scores.
4. Save weights and choose best model type. Use the same segmentation algo during inference and feed individual chars. No rotations during inference.

5. Concatenate predictions in order to generate final output.

Model	CNN	Resnet-18	SqueezeNet-Mini	MobileNet	E(2)CNN
Captcha recognition accuracy	0.18	0.30	0.25	0.27	0.16
Character recognition accuracy	0.72	0.83	0.78	0.79	0.79

Models trained

Resnet-18

CNN:

Simple 2 Convolution, 1 maxpool + fc model

SqueezeNet-Mini:

Use known architecture with **Fire blocks** (1x1 cat 3x3), BUT reduce stride size and remove one maxpool layer as img is small

MobileNet:

Use Inverted Residuals and expansions, reduce expansions due to small img size

E(2)CNN:

Equivariant neural networks that generalize over translations, rotations and reflections, improving recognition with rotated captchas

CAPTCHA Generation

We attempted a GAN-based CAPTCHA generator with two approaches.

The first approach trained a generic GAN solely reliant on the dataset's CAPTCHAs for training. However, the results were unreliable, likely due to the wide and inconsistent sizing of the training data.



Generated CAPTCHAs at epoch 80 using a generic GAN

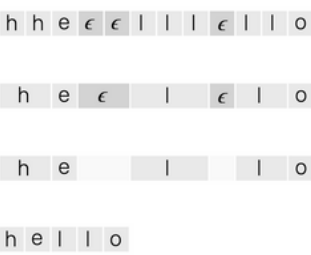
The second approach developed a CAPTCHA generator built upon a AC-GAN-based character generator using self-attention and spectral normalisation for stability. The GAN was pre-trained to be able to generate realistic alphanumeric characters, before these characters were put together procedurally to generate realistic CAPTCHAs.



Other Approach Explored

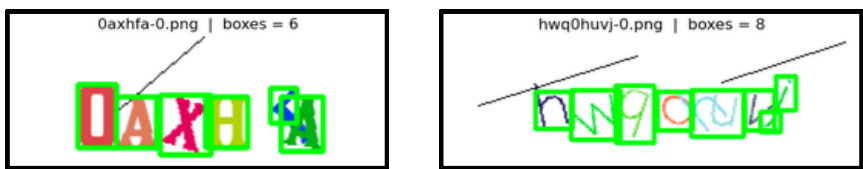
OCR-style with BiLSTM CTC

- Common, sequence-based approach for OCR and handwriting recognition, no need for segmentation
- Separates characters with a special "blank" token



Color segmentation by K-clustering

- Color-based character segmentation with K-means
- Geometry and color aware splitting of fused boxes
- Character extraction and standardisation for training



Transfer learning with FasterRCNN

- PyTorch's pre-trained FasterRCNN with a ResNet-50-FPN backbone
- Model for locating and classifying characters in CAPTCHA image
- DBSCAN to label dataset in COCO format for training

